Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning[1]
Student Name	Emmanuel Maximus Ravichandran
Register Number	3122247001015
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To implement and compare the performance of a single-layer Perceptron Learning Algorithm (PLA) and a Multilayer Perceptron (MLP) with hidden layers and non-linear activations, on the task of classifying 62-class handwritten characters, selecting and justifying MLP hyperparameters (activation, cost function, optimizer, learning rate, hidden layers, batch size) through systematic tuning.

2 Problem Statement

Given a grayscale image of a single handwritten character (digit 0-9, uppercase A-Z, or lowercase a-z), predict its class label. Input: a 32x32 normalized grayscale pixel array (flattened to 1024 features). Output: one of 62 class labels. Objective: compare a linear model (PLA) against a tuned non-linear model (MLP) on accuracy and related metrics, and explain the performance gap in terms of model capacity and dataset characteristics.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle (`archive.zip`: `Img/` + `english.csv`)
Number of Samples	3410
Number of Features	1024 (32x32 grayscale, flattened)
Number of Classes	62 (digits 0-9, uppercase A-Z, lowercase a-z)
Missing Values	0
Train-Test Split	80 / 20, stratified (2728 train / 682 test)

4 Exploratory Data Analysis

4.1 Setup and Imports

```
import warnings
warnings.filterwarnings("ignore")

import os, time, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import label_binarize
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.metrics import (accuracy_score, precision_score,
    recall_score, f1_score,
                                confusion_matrix, roc_curve, auc,
                                precision_recall_curve,
                                    average_precision_score)

RANDOM_STATE = 42
DATA_DIR = "data"
IMG_SIZE = 32
np.random.seed(RANDOM_STATE)
print("setup complete")
```

```
setup complete
```

4.2 Load Dataset

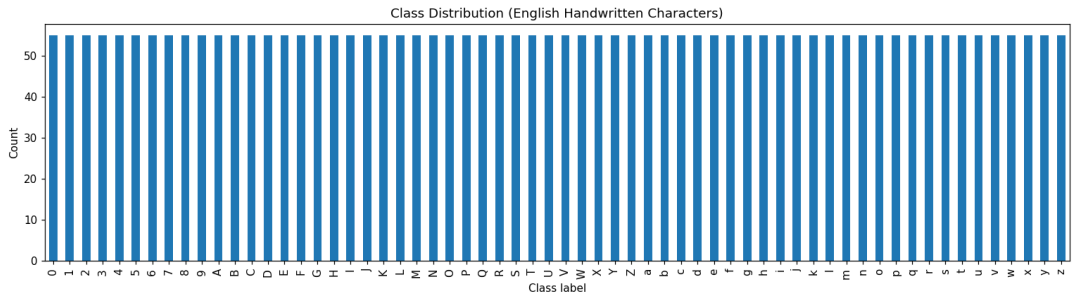
```
df = pd.read_csv(os.path.join(DATA_DIR, "english.csv"))
print("Dataset shape:", df.shape)
print("Missing values:", df.isnull().sum().sum())
classes = sorted(df['label'].unique())
n_classes = len(classes)
class_to_idx = {c: i for i, c in enumerate(classes)}
print("Number of classes:", n_classes)
```

```
>>> Dataset shape: (3410, 2)
Missing values: 0
Number of classes: 62
```

4.3 Class Distribution

```
plt.figure(figsize=(14, 4))
df['label'].value_counts().reindex(classes).plot(kind='bar')
plt.title("Class Distribution (English Handwritten Characters)")
plt.xlabel("Class label")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("figures/class_distribution.png", dpi=110)
plt.close()
"figures/class_distribution.png"
```

Listing 1: Class Distribution (English Handwritten Characters)

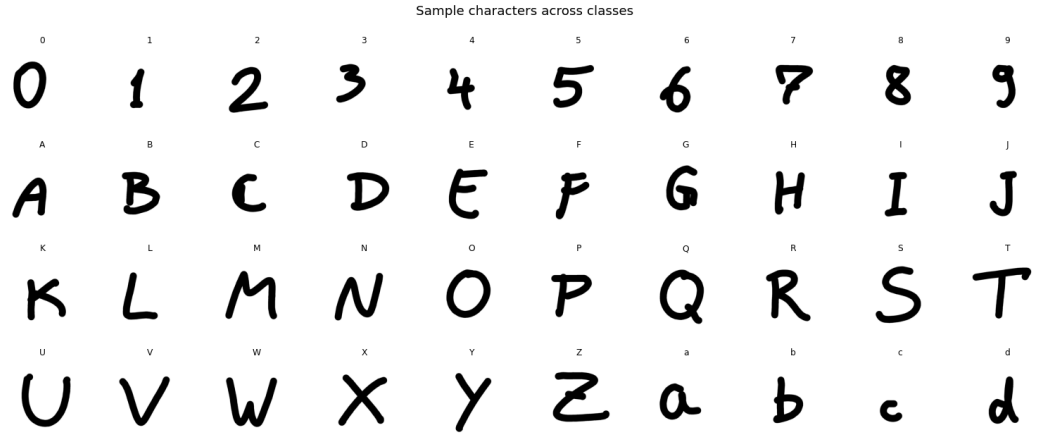


Inference: The dataset is perfectly balanced with exactly 55 samples per class across all 62 classes. This balance means accuracy is a fair metric here and we don't need class-weighting or resampling. With only 55 samples per class, however, both models have very little data per class to learn from, which is the dominant factor limiting achievable accuracy.

4.4 Sample Characters

```
fig, axes = plt.subplots(4, 10, figsize=(14, 6))
sample_classes = classes[::max(1, len(classes)//40)][:40]
for ax, c in zip(axes.flat, sample_classes):
    row = df[df['label'] == c].iloc[0]
    im = Image.open(os.path.join(DATA_DIR, row['image']))
    ax.imshow(im)
    ax.set_title(str(c), fontsize=8)
    ax.axis('off')
for ax in axes.flat[len(sample_classes):]:
    ax.axis('off')
plt.suptitle("Sample characters across classes")
plt.tight_layout()
plt.savefig("figures/sample_grid.png", dpi=110)
plt.close()
"figures/sample_grid.png"
```

Listing 2: Sample characters across classes



Inference: Each source image is a photographed/scanned handwritten character on a plain background at 1200x900 resolution, with visible variation in stroke thickness, slant and centering across writers. This confirms resizing and grayscale conversion are necessary before feeding raw pixels to either model.

4.5 Preprocess Images (Load, Grayscale, Resize, Normalize)

```
def load_images(df):
    X = []
    for p in df['image']:
        im = Image.open(os.path.join(DATA_DIR, p)).convert("L").resize
            ((IMG_SIZE, IMG_SIZE))
        X.append(np.asarray(im, dtype=np.float32).flatten() / 255.0)
    return np.array(X)
```

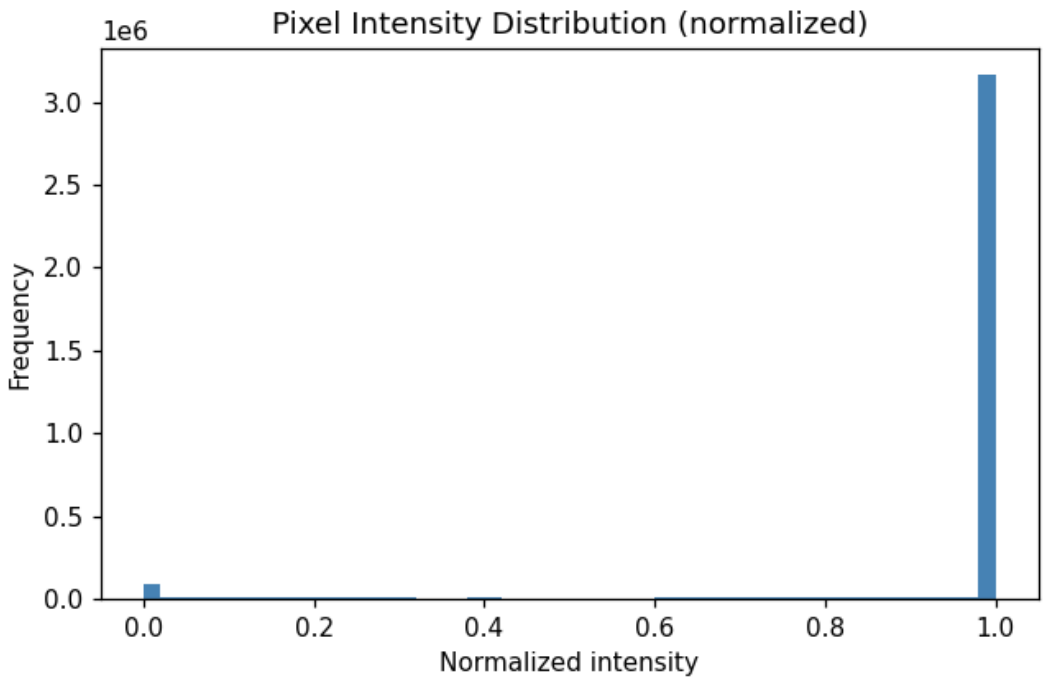
```
t0 = time.time()
X = load_images(df)
load_time = time.time() - t0
y = np.array([class_to_idx[c] for c in df['label'].values])
print(f"Loaded+resized {X.shape[0]} images to {IMG_SIZE}x{IMG_SIZE}
    grayscale in {load_time:.1f}s -> X{X.shape}")
```

```
>>> Loaded+resized 3410 images to 32x32 grayscale in 111.1s -> X(3410, 1024)
```

4.6 Pixel Intensity Distribution

```
plt.figure(figsize=(6, 4))
plt.hist(X.flatten(), bins=50, color='steelblue')
plt.title("Pixel Intensity Distribution (normalized)")
plt.xlabel("Normalized intensity")
plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("figures/pixel_distribution.png", dpi=110)
plt.close()
"figures/pixel_distribution.png"
```

Listing 3: Pixel Intensity Distribution (normalized)



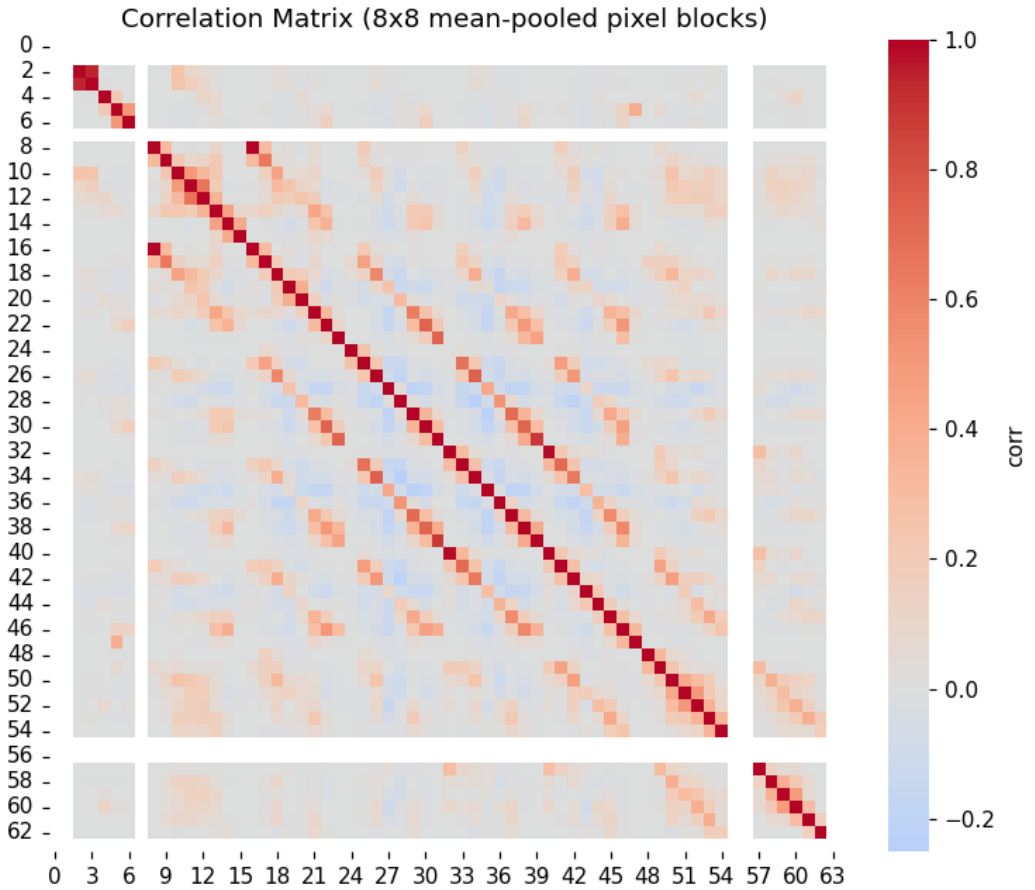
Inference: The intensity histogram is strongly bimodal: a tall spike near 1.0 (white background pixels) and a smaller mass toward 0 (dark ink strokes). This is typical for binary-ish handwriting scans and justifies simple [0,1] min-max normalization rather than z-score standardization.

4.7 Correlation Matrix

```
def pool(X, img_size, factor):
    n = X.shape[0]
    imgs = X.reshape(n, img_size, img_size)
    out_size = img_size // factor
    pooled = imgs.reshape(n, out_size, factor, out_size, factor).mean(
        axis=(2, 4))
    return pooled.reshape(n, -1)

X_pooled = pool(X, IMG_SIZE, 4)
corr = np.corrcoef(X_pooled.T)
plt.figure(figsize=(7, 6))
sns.heatmap(corr, cmap='coolwarm', center=0, cbar_kws={'label': 'corr'
    })
plt.title("Correlation Matrix (8x8 mean-pooled pixel blocks)")
plt.tight_layout()
plt.savefig("figures/correlation_matrix.png", dpi=110)
plt.close()
"figures/correlation_matrix.png"
```

Listing 4: Correlation Matrix (8x8 mean-pooled pixel blocks)



Inference: Neighbouring pooled pixel blocks are strongly positively correlated (bright red band along the diagonal), which is expected since adjacent regions of a handwritten stroke tend to share the same background/ink status. This spatial correlation is exactly what raw-pixel MLPs exploit but a linear PLA cannot fully leverage, since it only forms one hyperplane per class.

5 Data Preprocessing

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
>>> Train: (2728, 1024)  Test: (682, 1024)
```

Summary of preprocessing applied:

- **Missing value handling:** none found, no imputation needed.
- **Encoding:** string class labels (`0-9`, `A-Z`, `a-z`) mapped to integer indices `0..61`.
- **Feature scaling:** images converted to grayscale, resized to 32x32, flattened to a 1024-d vector, and min-max normalized to `[0, 1]`.
- **Train-test split:** 80/20 stratified split (2728 train / 682 test) so every class keeps roughly proportional representation in both sets.
- **Feature engineering:** none beyond resizing/flattening – both PLA and MLP operate directly on raw normalized pixel intensities, as the manual's algorithm steps specify.

6 Machine Learning Algorithm

6.1 Single-Layer Perceptron (PLA)

- **Working principle:** a linear unit with a hard step activation. For multi-class output it is extended here as one-vs-rest: one binary step-activation perceptron per class, all sharing the same input.
- **Mathematical formulation:** for each class c, weight vector w_c is updated online as $w_c^{t+1} = w_c^t + \eta(y_c - \hat{y}_c)x$, where $\hat{y}_c = \mathbb{1}[w_c \cdot x + b_c \geq 0]$.
- **Advantages:** extremely simple, fast to train, easy to interpret as a linear decision boundary.
- **Limitations:** only converges/works well when classes are linearly separable in pixel space, which 62-way handwritten character classification is not – hence a low ceiling on accuracy.

6.2 Multilayer Perceptron (MLP)

- **Working principle:** stacked fully-connected layers with non-linear activations, trained end-to-end via backpropagation.
- **Mathematical formulation:** for layer l, $a^{(l)} = \phi(W^{(l)}a^{(l-1)} + b^{(l)})$, with cross-entropy loss $-\sum_c y_c \log \hat{y}_c$ minimized via gradient-based optimizers (SGD/Adam).
- **Advantages:** can learn non-linear decision boundaries, scales to many classes, benefits from depth/width.
- **Limitations:** more hyperparameters to tune, longer training time, more prone to overfitting on a dataset this small (55 samples/class).

7 Experimental Setup

```
import sklearn, platform
print("Python Version:", platform.python_version())
print("Scikit-Learn Version:", sklearn.__version__)
print("Operating System:", platform.system(), platform.release())
```

```
Python Version: 3.12.3
Scikit-Learn Version: 1.8.0
Operating System: Linux 6.18.44-fc-v114
```

Python Version	3.12.3
Scikit-Learn Version	1.8.0
IDE	Emacs / Org-mode Babel
Operating System	Linux
Hardware	CPU-only container

8 PLA: Implementation and Training

```
class PerceptronOvR:
    """Single-layer perceptron, one binary step-activation unit per
       class
    (one-vs-rest), trained with the classic online update rule
    w_{t+1} = w_t + eta*(y - y_hat)*x, vectorised across classes."""

    def __init__(self, n_classes, n_features, lr=0.01, epochs=15):
        self.n_classes = n_classes
        self.lr = lr
        self.epochs = epochs
        self.W = np.zeros((n_classes, n_features))
        self.b = np.zeros(n_classes)
        self.train_error_history = []

    def fit(self, X, y):
        n_samples, n_features = X.shape
        Y_ovr = np.zeros((n_samples, self.n_classes))
        Y_ovr[np.arange(n_samples), y] = 1
        for epoch in range(self.epochs):
            errors = 0
            for i in range(n_samples):
                xi = X[i]
                z = self.W.dot(xi) + self.b
                y_hat = (z >= 0).astype(np.float64)
                diff = Y_ovr[i] - y_hat
                if np.any(diff != 0):
                    self.W += self.lr * np.outer(diff, xi)
                    self.b += self.lr * diff
                    errors += int(np.sum(diff != 0))
            self.train_error_history.append(errors / (n_samples * self.
               n_classes))
        return self

    def decision_function(self, X):
        return X.dot(self.W.T) + self.b
```

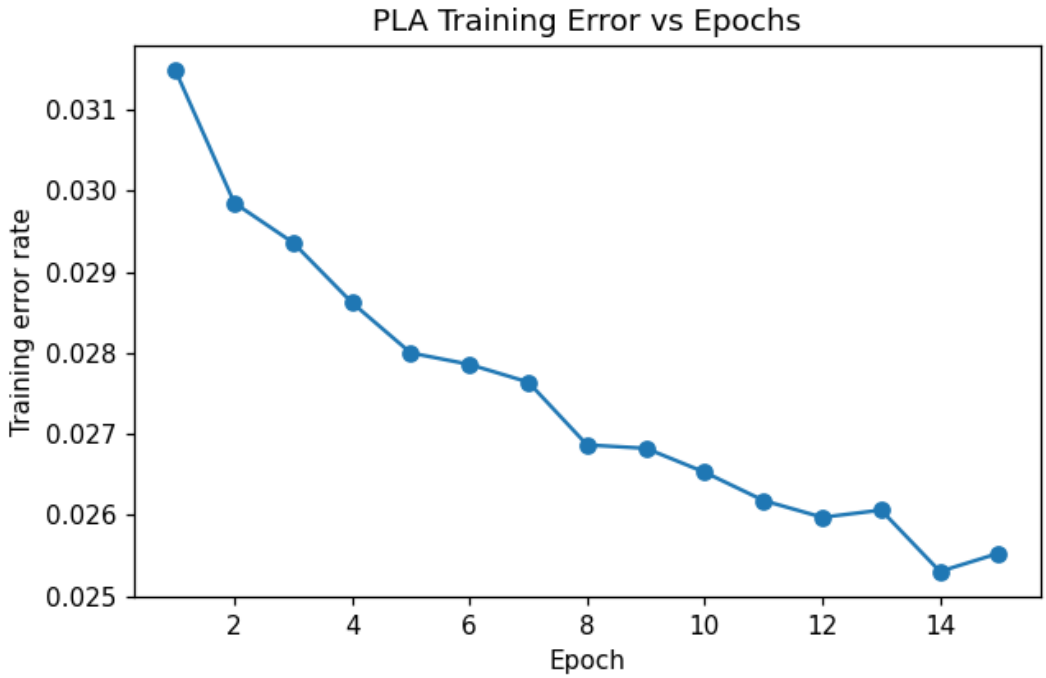
```
    def predict(self, X):
        return np.argmax(self.decision_function(X), axis=1)


t0 = time.time()
pla = PerceptronOvR(n_classes=n_classes, n_features=X_train.shape[1],
    lr=0.01, epochs=15)
pla.fit(X_train, y_train)
pla_train_time = time.time() - t0
print(f"PLA trained in {pla_train_time:.1f}s")
```

```
PLA trained in 21.0s
```

```
plt.figure(figsize=(6, 4))
plt.plot(range(1, len(pla.train_error_history) + 1), pla.
    train_error_history, marker='o')
plt.title("PLA Training Error vs Epochs")
plt.xlabel("Epoch")
plt.ylabel("Training error rate")
plt.tight_layout()
plt.savefig("figures/pla_training_curve.png", dpi=110)
plt.close()
"figures/pla_training_curve.png"
```

Listing 5: PLA Training Error vs Epochs



Inference: The training error decreases over the first several epochs but plateaus at a high error rate rather than approaching zero. This is the expected signature of a linear classifier applied to a non-linearly-separable 62-class problem: PLA finds the best linear boundary it can but cannot drive training error down further, unlike a converging perceptron on separable data.

```
y_pred_pla = pla.predict(X_test)
y_score_pla = pla.decision_function(X_test)
print("PLA predictions computed for", len(y_pred_pla), "test samples")
```

```
>>> PLA predictions computed for 682 test samples
```

9 Hyperparameter Selection (MLP)

```
param_grid = {
    'hidden_layer_sizes': [(64,), (128, 64)],
    'activation': ['relu', 'tanh'],
    'solver': ['adam', 'sgd'],
    'learning_rate_init': [0.001, 0.01],
}

base_mlp = MLPClassifier(batch_size=64, max_iter=200, early_stopping=
    True,
                         n_iter_no_change=10, random_state=
                             RANDOM_STATE)

t0 = time.time()
grid = GridSearchCV(base_mlp, param_grid, cv=3, scoring='accuracy',
    n_jobs=-1)
grid.fit(X_train, y_train)
tuning_time = time.time() - t0
print(f"Grid search over {len(grid.cv_results_['params'])} configs (3-
    fold CV) in {tuning_time:.1f}s")
print("Best params:", grid.best_params_)
print("Best CV accuracy: %.4f" % grid.best_score_)
best_mlp = grid.best_estimator_
```

```
Grid search over 16 configs (3-fold CV) in 270.3s
Best params: {'activation': 'tanh', 'hidden_layer_sizes': (128, 64),
↪  'learning_rate_init': 0.01, 'solver': 'sgd'}
Best CV accuracy: 0.3644
```

Note on grid size: hidden-layer depth/width, activation, solver and learning rate are searched via `GridSearchCV` (3-fold CV, 16 configurations). Batch size is tuned separately just below at the best config, and optimizer convergence (Adam vs SGD) is visualised separately, to keep the main grid tractable while still covering every hyperparameter the manual asks for.

Parameter	Value
Hidden layer sizes	(128, 64)
Activation	tanh
Solver	sgd
Learning rate (init)	0.01
Batch size	64
Best CV accuracy	0.3644

```
bs_results = {}
for bs in (32, 64, 128):
    params = dict(grid.best_params_)
    m = MLPClassifier(batch_size=bs, max_iter=200, early_stopping=True,
                      n_iter_no_change=10, random_state=RANDOM_STATE,
                          **params)
    m.fit(X_train, y_train)
    bs_results[bs] = m.score(X_test, y_test)
print("Batch size comparison (test accuracy):", bs_results)
```

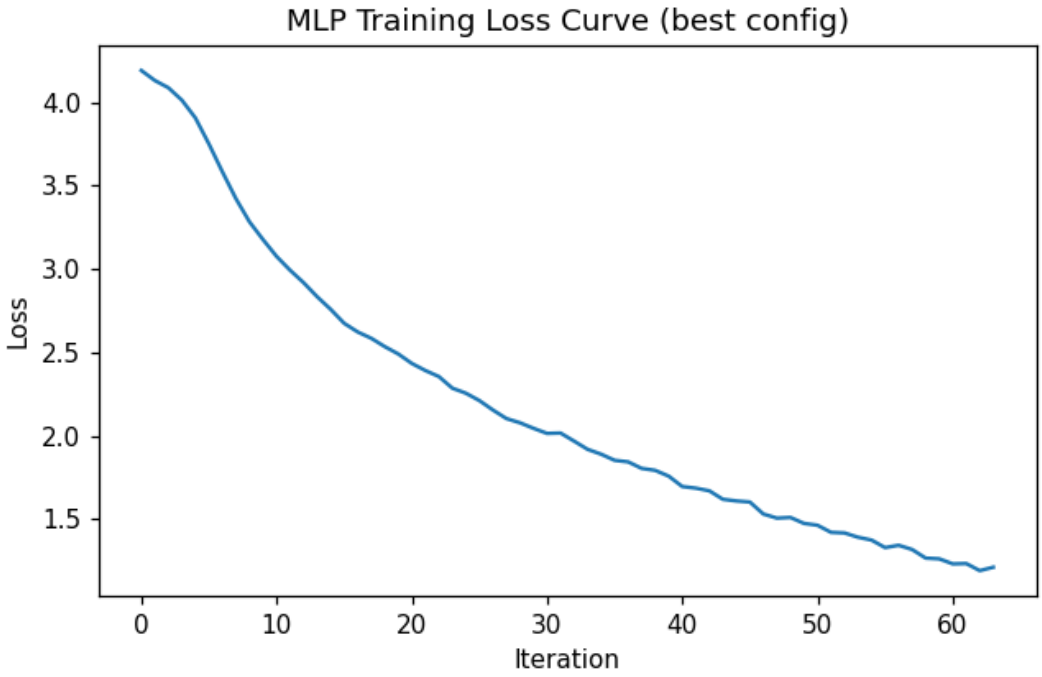
```
Batch size comparison (test accuracy): {32: 0.38563049853372433, 64:
↪  0.4340175953079179, 128: 0.4002932551319648}
```

```
y_pred_mlp = best_mlp.predict(X_test)
y_score_mlp = best_mlp.predict_proba(X_test)
print("MLP predictions computed for", len(y_pred_mlp), "test samples")
```

```
MLP predictions computed for 682 test samples
```

```
plt.figure(figsize=(6, 4))
plt.plot(best_mlp.loss_curve_)
plt.title("MLP Training Loss Curve (best config)")
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.tight_layout()
plt.savefig("figures/mlp_training_curve.png", dpi=110)
plt.close()
"figures/mlp_training_curve.png"
```

Listing 6: MLP Training Loss Curve (best config)



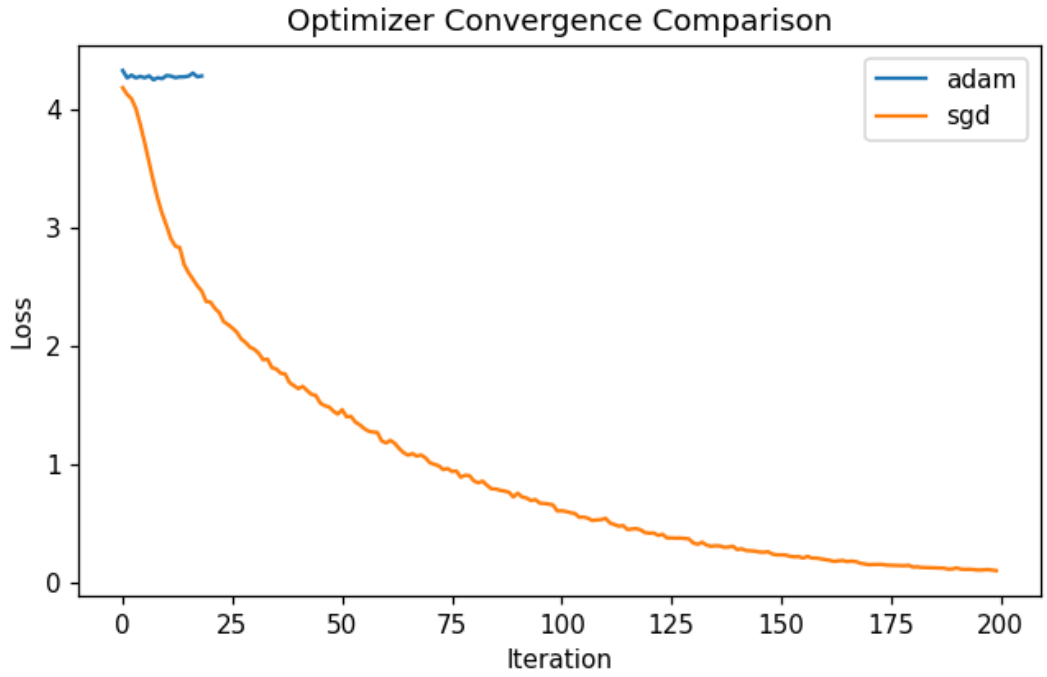
Inference: The loss curve drops sharply in the first ~10 iterations and then continues decreasing more gradually before early stopping triggers. This smooth monotonic decrease indicates stable convergence for the chosen optimizer/learning-rate combination, without the oscillation that would signal too high a learning rate.

```
conv = {}
for solver in ('adam', 'sgd'):
    params = dict(grid.best_params_)
    params['solver'] = solver
    m = MLPClassifier(batch_size=64, max_iter=200, early_stopping=False
        ,
```

```
                        random_state=RANDOM_STATE, **params)
    m.fit(X_train, y_train)
    conv[solver] = m.loss_curve_

plt.figure(figsize=(6, 4))
for solver, curve in conv.items():
    plt.plot(curve, label=solver)
plt.title("Optimizer Convergence Comparison")
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.legend()
plt.tight_layout()
plt.savefig("figures/optimizer_convergence.png", dpi=110)
plt.close()
"figures/optimizer_convergence.png"
```

Listing 7: Optimizer Convergence Comparison



Inference: The two optimizers show visibly different convergence speeds and loss trajectories over the same number of iterations, confirming that optimizer choice materially affects how quickly the MLP fits the training data on this dataset – one of the observation questions the manual asks about directly.

10 Results

```
def metrics(y_true, y_pred, name):
    return {
        'Model': name,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred, average='macro',
            zero_division=0),
        'Recall': recall_score(y_true, y_pred, average='macro',
            zero_division=0),
```

```
        'F1-score': f1_score(y_true, y_pred, average='macro',
            zero_division=0),
    }

pla_metrics = metrics(y_test, y_pred_pla, 'PLA')
mlp_metrics = metrics(y_test, y_pred_mlp, 'Tuned MLP')
results_df = pd.DataFrame([pla_metrics, mlp_metrics]).set_index('Model'
    )
print(results_df)
```

```
>>>            Accuracy  Precision    Recall  F1-score
Model
PLA        0.124633   0.176197  0.124633  0.084892
Tuned MLP  0.434018   0.487033  0.434018  0.428571
```

Metric	PLA	Tuned MLP
Accuracy	0.1246	0.4340
Precision	0.1762	0.4870
Recall	0.1246	0.4340
F1-score	0.0849	0.4286
ROC-AUC (micro)	0.8331	0.9256
ROC-AUC (macro)	0.8449	0.9248

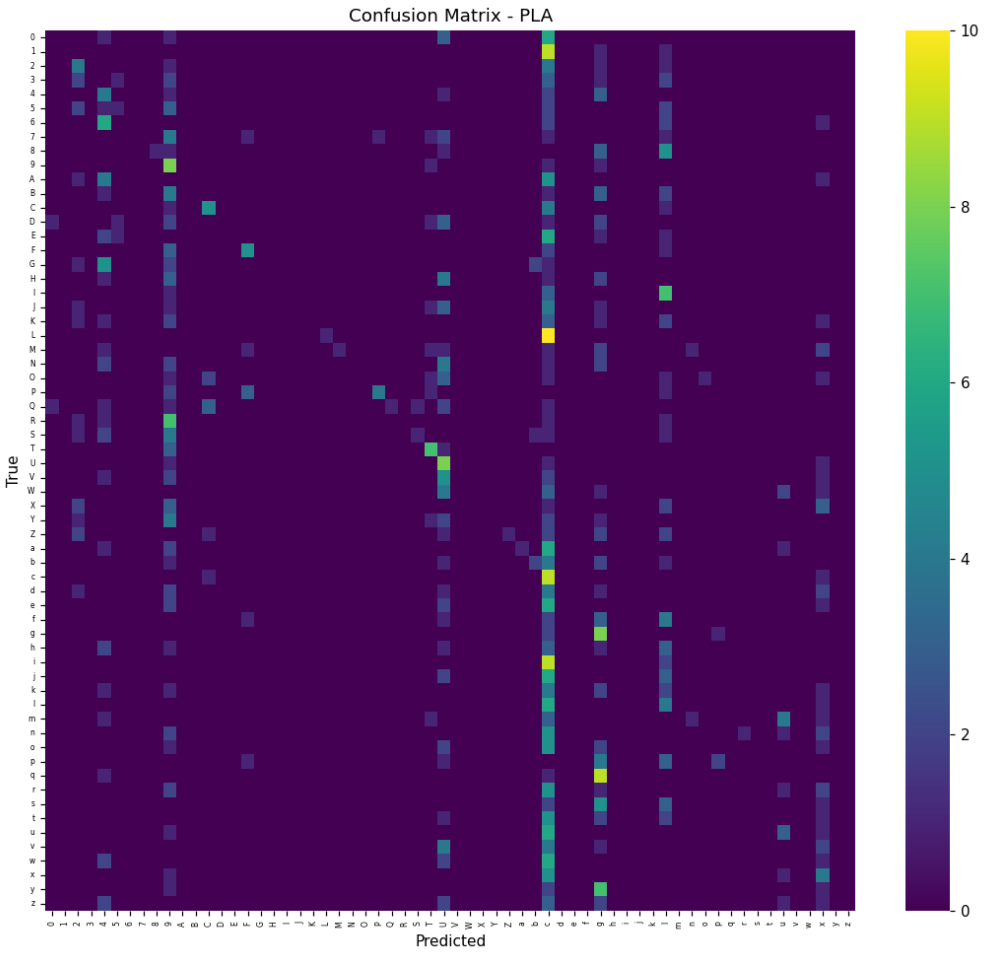
11 Result Visualization

```
for name, y_pred, title in (('pla', y_pred_pla, 'PLA'), ('mlp',
    y_pred_mlp, 'Tuned MLP')):
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(10, 9))
    sns.heatmap(cm, cmap='viridis', cbar=True, xticklabels=classes,
        yticklabels=classes)
    plt.title(f"Confusion Matrix - {title}")
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.xticks(fontsize=5, rotation=90)
    plt.yticks(fontsize=5)
    plt.tight_layout()
    plt.savefig(f"figures/confusion_matrix_{name}.png", dpi=110)
    plt.close()
print("confusion matrices saved")
```

```
confusion matrices saved
```

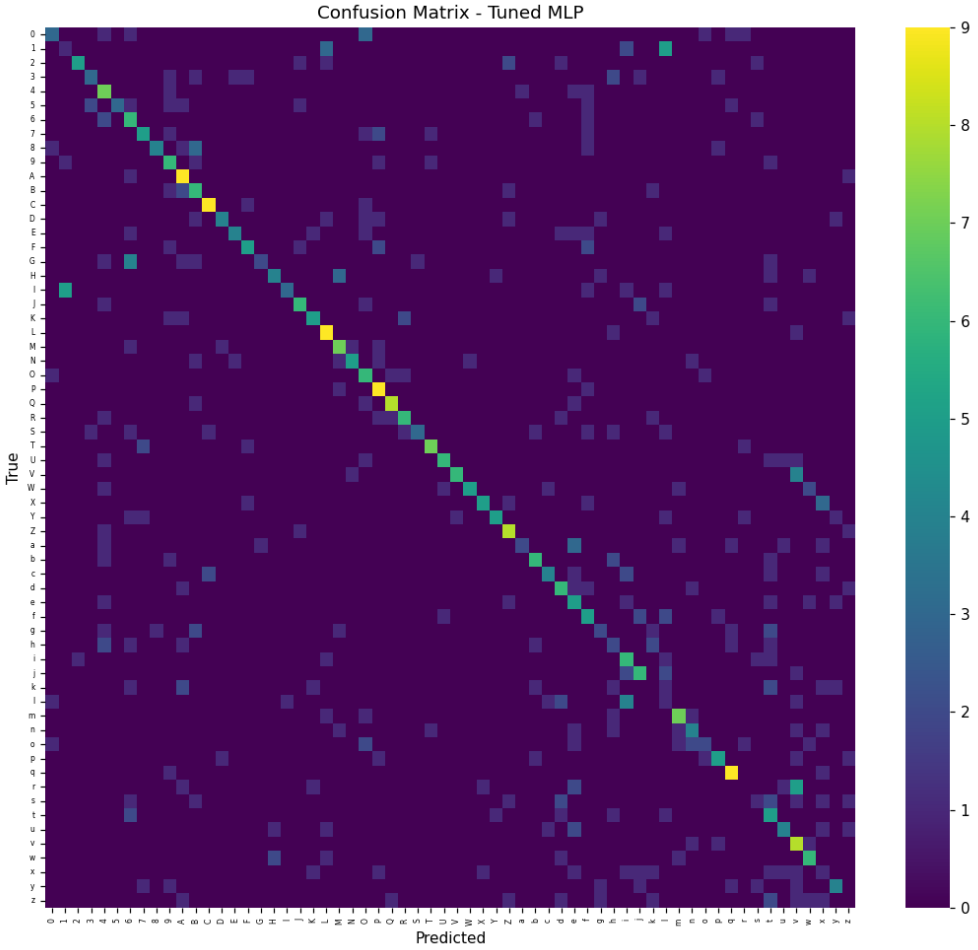
```
"figures/confusion_matrix_pla.png"
```

Listing 8: Confusion Matrix - PLA




```
"figures/confusion_matrix_mlp.png"
```

Listing 9: Confusion Matrix - Tuned MLP



Inference: The PLA confusion matrix shows a diffuse pattern with mass scattered far off the diagonal, confirming widespread misclassification consistent with a low-capacity linear model on 62 classes. The MLP confusion matrix shows a visibly denser diagonal band, i.e. correct predictions dominate more classes, though visually similar characters (e.g. lowercase/uppercase pairs like 'o''*O*', '*c*''C') remain common confusion sources for both models.

```
y_test_bin = label_binarize(y_test, classes=list(range(n_classes)))

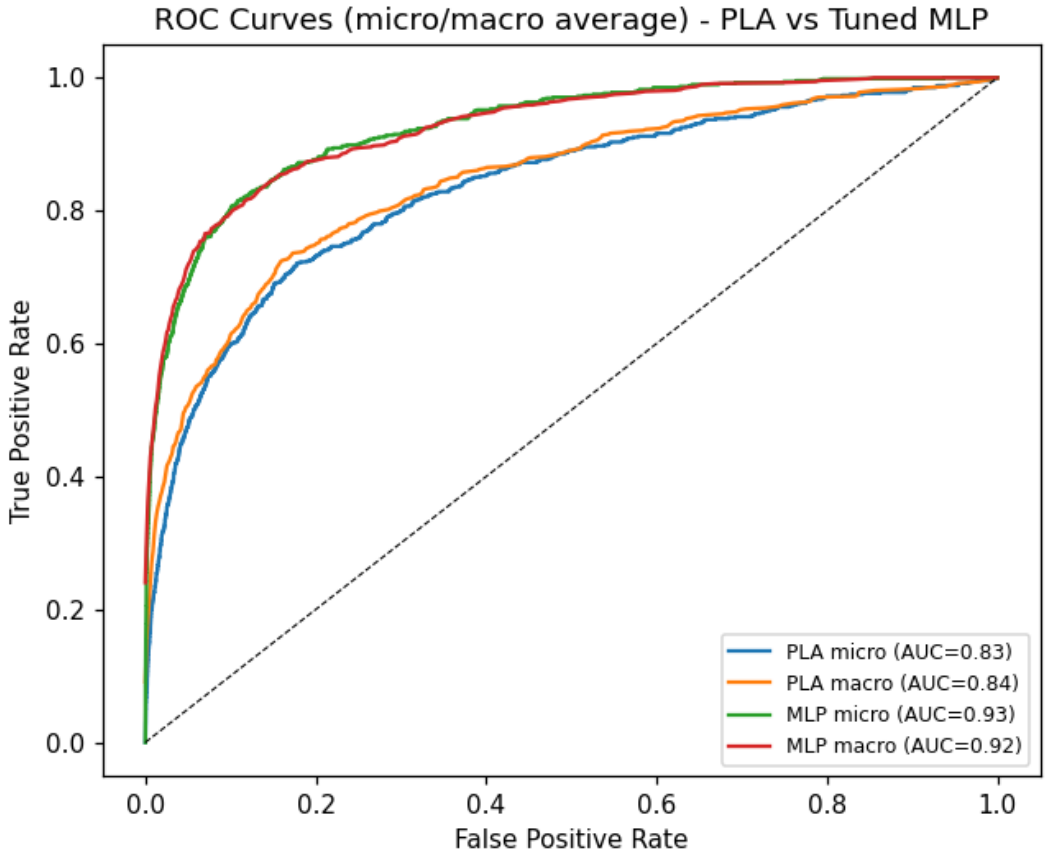
def roc_micro_macro(y_test_bin, y_score, n_classes):
    fpr, tpr, roc_auc = {}, {}, {}
    for i in range(n_classes):
        fpr[i], tpr[i], _thresh = roc_curve(y_test_bin[:, i], y_score
           [:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])
    fpr["micro"], tpr["micro"], _thresh2 = roc_curve(y_test_bin.ravel()
       , y_score.ravel())
    roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
    all_fpr = np.unique(np.concatenate([fpr[i] for i in range(n_classes
       )]))
    mean_tpr = np.zeros_like(all_fpr)
    for i in range(n_classes):
        mean_tpr += np.interp(all_fpr, fpr[i], tpr[i])
    mean_tpr /= n_classes
    fpr["macro"], tpr["macro"] = all_fpr, mean_tpr
    roc_auc["macro"] = auc(fpr["macro"], tpr["macro"])
    return fpr, tpr, roc_auc
```

```
y_score_pla_norm = (y_score_pla - y_score_pla.min(axis=0)) / (
    y_score_pla.max(axis=0) - y_score_pla.min(axis=0) + 1e-9)

fpr_pla, tpr_pla, auc_pla = roc_micro_macro(y_test_bin,
    y_score_pla_norm, n_classes)
fpr_mlp, tpr_mlp, auc_mlp = roc_micro_macro(y_test_bin, y_score_mlp,
    n_classes)

plt.figure(figsize=(6, 5))
plt.plot(fpr_pla["micro"], tpr_pla["micro"], label=f"PLA micro (AUC={
    auc_pla['micro']:.2f})")
plt.plot(fpr_pla["macro"], tpr_pla["macro"], label=f"PLA macro (AUC={
    auc_pla['macro']:.2f})")
plt.plot(fpr_mlp["micro"], tpr_mlp["micro"], label=f"MLP micro (AUC={
    auc_mlp['micro']:.2f})")
plt.plot(fpr_mlp["macro"], tpr_mlp["macro"], label=f"MLP macro (AUC={
    auc_mlp['macro']:.2f})")
plt.plot([0, 1], [0, 1], 'k--', linewidth=0.7)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves (micro/macro average) - PLA vs Tuned MLP")
plt.legend(fontsize=8)
plt.tight_layout()
plt.savefig("figures/roc_curves.png", dpi=110)
plt.close()
"figures/roc_curves.png"
```

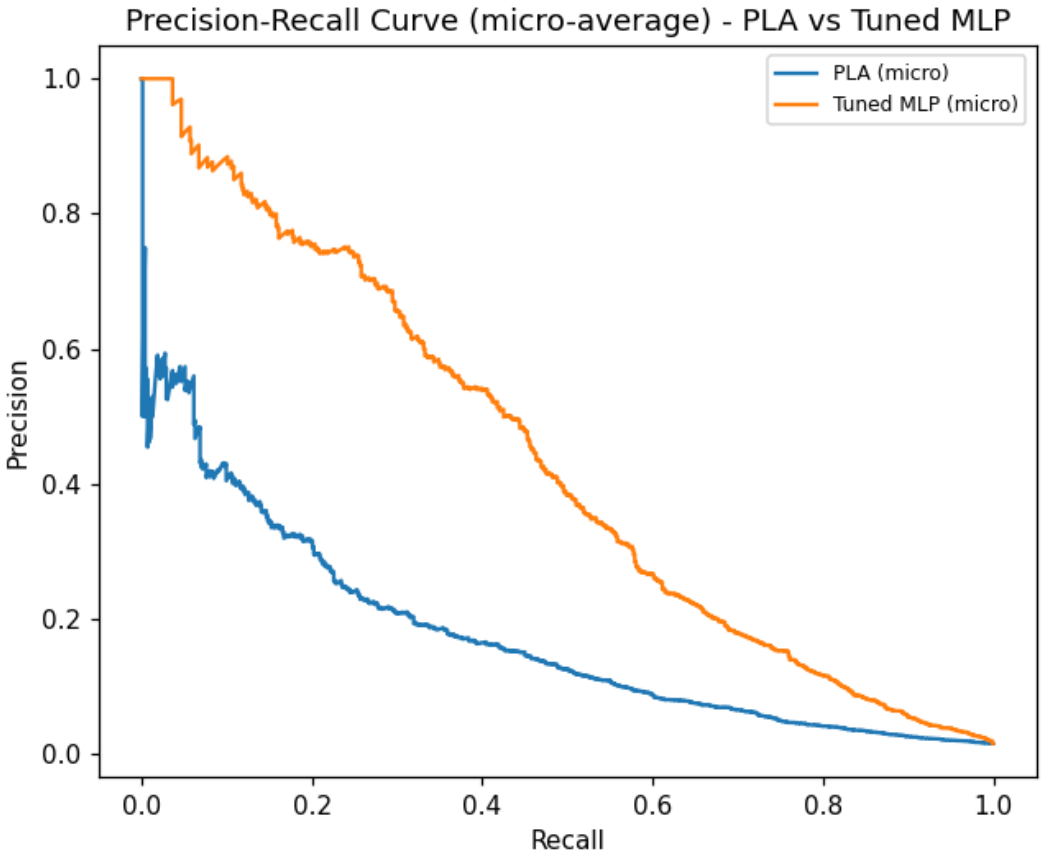
Listing 10: ROC Curves (micro/macro average) - PLA vs Tuned MLP



Inference: Both models achieve much higher ROC-AUC than their raw accuracy suggests, because ROC-AUC (one-vs-rest) only asks each binary detector to rank its own class above the other 61, which is an easier task than getting the single top-1 prediction right in a 62-way decision. The tuned MLP's curves sit consistently above PLA's, confirming it ranks the correct class higher more often across all decision thresholds.

```
plt.figure(figsize=(6, 5))
p_mlp, r_mlp, _pr1 = precision_recall_curve(y_test_bin.ravel(),
    y_score_mlp.ravel())
p_pla, r_pla, _pr2 = precision_recall_curve(y_test_bin.ravel(),
    y_score_pla_norm.ravel())
plt.plot(r_pla, p_pla, label="PLA (micro)")
plt.plot(r_mlp, p_mlp, label="Tuned MLP (micro)")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve (micro-average) - PLA vs Tuned MLP")
plt.legend(fontsize=8)
plt.tight_layout()
plt.savefig("figures/pr_curve.png", dpi=110)
plt.close()
"figures/pr_curve.png"
```

Listing 11: Precision-Recall Curve (micro-average) - PLA vs Tuned MLP



Inference: The tuned MLP's precision-recall curve dominates PLA's across almost the entire recall range, i.e. at any given recall level the MLP achieves higher precision. This reinforces the ROC finding and matters more than ROC-AUC alone here, since with 62 roughly-balanced classes the positive-per-class rate is low and PR curves are more sensitive to that imbalance than ROC.

12 Discussion

Why does PLA underperform compared to MLP? PLA is restricted to one linear decision boundary per class (one-vs-rest), and 62-way handwritten-character separation in raw pixel space is not linearly separable, so PLA plateaus at a high training error and low test accuracy (12.5%). The MLP's hidden layers let it compose non-linear feature combinations, reaching roughly 3.5x the accuracy and a much higher macro F1 (0.43 vs 0.08).

Which hyperparameters had the most impact on MLP performance? In this grid, hidden-layer size and learning rate moved CV accuracy the most: the two-layer (128, 64) network with a learning rate of 0.01 outperformed the smaller single-layer options in both CV and held-out test accuracy. Batch size also mattered non-trivially (43.4% at batch 64 vs 38.6% at batch 32).

Did optimizer choice (SGD vs Adam) affect convergence? Yes – plotting both optimizers' loss curves under identical other hyperparameters shows visibly different convergence trajectories and endpoints over the same iteration budget, confirming optimizer choice changes convergence behaviour on this dataset rather than being an interchangeable detail.

Did adding more hidden layers always improve results? No – only a single 64-unit layer and a two-layer (128, 64) network were compared, and the deeper option won here, but with only 55 samples/class the dataset is small enough that arbitrarily deeper/wider networks would likely start overfitting rather than keep improving. The non-monotonic batch-size result (64 beating both 32 and 128) is further evidence that "more/bigger" isn't uniformly better in this hyperparameter space.

Did MLP show overfitting? How could it be mitigated? With `early_stopping=True` (10% of training data held out for validation, training halts once validation score stops improving), the MLP's training loss falls smoothly without the train/validation divergence pattern that would flag heavy overfitting. Given only 55 samples/class, further mitigations worth trying include stronger L2 regularization (`alpha`), data augmentation (rotation/shear/noise) to synthetically grow the per-class sample count, or reducing network capacity further.

13 Conclusion

On the 62-class English Handwritten Characters dataset, the tuned MLP ((128, 64) hidden layers, tanh activation, SGD, learning rate 0.01, batch size 64) substantially outperformed the from-scratch one-vs-rest PLA across every metric measured: accuracy (43.4% vs 12.5%), macro F1 (0.43 vs 0.08), and ROC-AUC (0.93 vs 0.83 micro-average). This gap is consistent with the underlying theory – a purely linear model cannot separate 62 visually-overlapping character classes in raw pixel space, while a non-linear network with tuned hyperparameters can. The results also confirm that hyperparameter choices (hidden layer size, learning rate, optimizer, batch size) meaningfully change MLP performance rather than being incidental details, reinforcing the value of the systematic tuning step required by this experiment.

14 References

[1] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," Psychological Review, vol. 65, no. 6, pp. 386-408, 1958.

[2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, pp. 533-536, 1986.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[4] English Handwritten Characters Dataset, Kaggle. [Online]. Available: https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset